

Latency, Throughput & the Numbers Every Engineer Knows

Part I · Foundations — the math that decides one server or a thousand.

LEARNING OBJECTIVES

- Define latency and throughput precisely, and explain how they interact.
- Explain why latency is a **distribution** (percentiles), and why the **tail** (p99) rules user experience.
- Explain why latency explodes as a system approaches full utilization — and why you never run at 100%.
- Recall the **numbers every engineer should know** and use them in rough calculations.
- Perform **back-of-the-envelope estimation**: QPS, storage, bandwidth, and server count from a user count.
- Translate those numbers into architecture decisions (cache, CDN, sharding, queues).

REAL-WORLD ANALOGY — PLANNING A STADIUM

Before building a stadium, engineers do napkin math: how many fans (users), arriving how fast (throughput), how long to pass through a gate (latency), so how many gates and how wide the roads (bandwidth) and how big the parking lot (storage)? They don't compute it to the exact person — they get the **order of magnitude** right, because the difference between planning for 5,000 and 50,000 fans changes everything. **Back-of-the-envelope estimation** is that napkin math for software: quick, rough, and decisive.

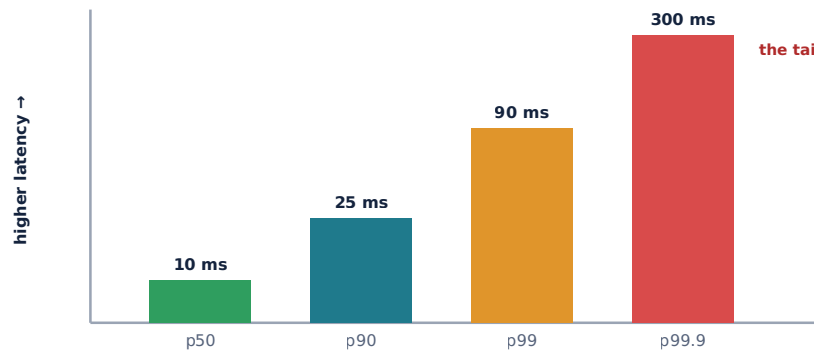
5.1 Why numbers come before architecture

Chapter 1 said non-functional requirements — scale, latency, availability — shape the design. This chapter is the toolkit that turns a vague “we'll have a lot of users” into concrete figures: this many requests per second, this much storage per year, this much bandwidth. Those figures tell you the **shape** of the system before you draw it. Being *roughly right* (one server vs one thousand) matters far more than being precisely wrong. Estimation is the single most useful skill in a design interview and in real planning.

5.2 Latency is a distribution, not a number

Latency is how long one request takes. But no two requests take the same time, so a single “average” hides the truth. Real systems describe latency with **percentiles**:

- **p50** (median): half of requests are faster than this. The “typical” experience.
- **p95 / p99**: 95% / 99% of requests are faster than this. The **tail** — your unluckiest users.
- **p99.9**: the worst 0.1%. At scale, that's still millions of requests.



Bar heights are illustrative; the shape is the point — the **typical** request is fast, but the tail is many times slower. That tail is what frustrated users actually feel.

KEY INSIGHT — “TAIL AT SCALE”

Modern requests often fan out to many services (recall the Netflix “press play” trace). If one page needs 50 internal calls, and each has a p99 of 100 ms, then the *page* is very likely to hit at least one slow call — so the **whole page is as slow as its slowest part**. This is why big companies obsess over p99 and p99.9, not the average. Cutting tail latency is often the highest-impact performance work you can do.

5.3 Throughput and its limits

Throughput is how many requests the system handles per second (often written **QPS** — queries per second, or RPS). Where latency is about one request, throughput is about the whole flow.

A useful rule of thumb links the two — **Little’s Law**: the number of requests being handled at once (concurrency) equals throughput multiplied by latency.

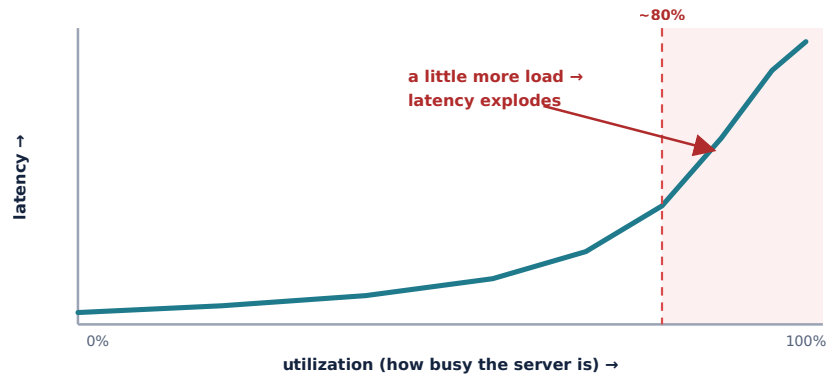
$$\text{concurrency} = \text{throughput} \times \text{latency}$$

e.g. 1,000 requests/sec x 0.2 sec each = 200 requests in flight at once.
 -> you need enough threads/connections/memory to hold ~200 at a time.

This tells you how much the system must juggle simultaneously, which drives thread pools, connection pools, and memory. Throughput is ultimately capped by the **slowest stage** in the path (the bottleneck) — a chain is only as fast as its narrowest link, so improving anything *except* the bottleneck won’t help.

5.4 How latency and throughput fight: the utilization curve

It’s tempting to run servers at 100% utilization to “get your money’s worth.” That is a trap. As a system gets busier, requests start **queuing** (waiting for a free worker), and once utilization nears its limit, latency doesn’t rise gently — it explodes.



KEY INSIGHT — LEAVE HEADROOM

Because latency spikes near full utilization, production systems are run with **spare capacity** (often targeting ~50–70% utilization). The extra headroom absorbs traffic bursts and keeps latency flat. Running “efficiently” at 95% utilization is how you get a system that is fast on Tuesday and collapses the moment traffic ticks up. Capacity planning (Chapter 51) is largely about choosing this headroom.

5.5 The numbers every engineer should know

Consolidating Chapters 2–4 into one reference. These are rounded orders of magnitude — memorize the *relationships*, not the exact digits.

Operation	Approx. time
Read from main memory (RAM)	~100 nanoseconds
SSD random read	~100 microseconds (~1,000× slower than RAM)
Network round trip, same data center	~0.5 milliseconds
Hard-disk seek	~10 milliseconds
Network round trip, across the world	~150 milliseconds

A few rough **capacity** figures to anchor estimates (very approximate — hardware varies):

Component	Rough single-node capacity
A cache (Redis/Memcached)	~100,000+ operations/sec
A relational database	~thousands of writes/sec, more for reads
A web/app server	~thousands of requests/sec (depends heavily on work)

And the two rounding tricks that make the math easy: there are **~86,400 seconds in a day** ≈ **100,000 (10⁵)**, and everything is powers of ten — KB, MB, GB, TB, PB each ×1,000.

5.6 The estimation method, step by step

Given “how many users,” produce the shape of the system in six quick moves:

Step	How
1. Users → actions	Start with daily active users (DAU) × actions per user per day.
2. Actions → QPS	Divide by ~100,000 (seconds/day). This gives average QPS.
3. Average → peak	Multiply average by ~2-3 for peak-hour traffic.
4. Read : write ratio	Split reads vs writes — this decides caching vs sharding emphasis.
5. Storage	bytes/item × items/day × days kept × replication factor.
6. Bandwidth	QPS × payload size per request.

5.7 A full worked example

SIZE A PHOTO-SHARING APP (100M DAILY ACTIVE USERS)

Assumptions: each user does ~20 feed reads/day and uploads ~0.2 photos/day (one every five days). A photo averages ~2 MB; each upload also writes ~1 KB of metadata. Keep data 5 years, replicate 3×.

Traffic (steps 1-3):

- Reads/day = $100\text{M} \times 20 = 2 \text{ billion}$ → $\div 100,000 = \sim\mathbf{20,000 \text{ read QPS}}$ average → peak $\times 3 \approx \mathbf{60,000}$.
- Writes/day = $100\text{M} \times 0.2 = 20 \text{ million}$ → $\div 100,000 = \sim\mathbf{200 \text{ write QPS}}$ average → peak $\approx \mathbf{600}$.
- Read : write $\approx \mathbf{100 : 1}$ → heavily read-dominated.

Storage (step 5):

- Photos: $20\text{M/day} \times 2 \text{ MB} = 40 \text{ TB/day}$ → $\times 365 \times 5 \approx 73 \text{ PB}$ → $\times 3$ replication $\approx \sim\mathbf{220 \text{ PB}}$.
- Metadata: $20\text{M/day} \times 1 \text{ KB} = 20 \text{ GB/day}$ → over 5 years $\approx \sim\mathbf{36 \text{ TB}}$.

Bandwidth (step 6): at read time, if each read serves ~200 KB of image: $20,000 \text{ reads/sec} \times 200 \text{ KB} \approx \sim\mathbf{4 \text{ GB/sec}}$ of egress.

WHAT THE NUMBERS JUST TOLD US

Without drawing one box, the arithmetic dictated the architecture: **100:1 reads** → a big **cache**; **220 PB of photos** → **object storage**, never a normal database; **4 GB/sec egress** → a **CDN** is mandatory; **36 TB of metadata** → a **sharded** database; and photos vs metadata are stored in *different* systems. The numbers designed the skeleton for us.

5.8 From numbers to architecture

What the numbers show	What it implies (previewing Part II-III)
Read-heavy (high read:write)	Add caching and read replicas .
Write-heavy or huge data	Shard/partition the database; consider queues to absorb writes.

Very large blobs (images/video)	Object storage + a CDN , kept separate from metadata.
High egress bandwidth	CDN to serve content near users and offload the origin.
Tight latency SLA (low p99)	Keep hot data in memory and near users; minimize round trips.
Spiky traffic	Queues to buffer, plus autoscaling and headroom.

Common mistakes

WATCH OUT

- Chasing false precision instead of the **order of magnitude** — the goal is “tens of thousands” not “18,432.”
- Judging latency by the **average** instead of p99/p99.9.
- Designing for average traffic and then dying at **peak**.
- Forgetting the **replication factor** and other overhead when estimating storage.
- Ignoring the **read:write ratio**, which decides caching vs sharding emphasis.
- Running systems near **100% utilization**, where latency explodes.
- Forgetting **bandwidth/egress** costs, especially for images and video.
- Lumping huge blobs and small metadata into one store instead of separating them.

Interview questions

1. A service has 50M DAU, each making 30 requests/day. Estimate average and peak QPS.
2. Why is p99 latency more important than the average? Explain “tail at scale.”
3. Why does latency spike as utilization approaches 100%? What utilization would you target, and why?
4. Estimate storage for 10M photos/day at 2 MB each, kept 3 years, replicated 3x.
5. State Little’s Law and use it: at 2,000 QPS with 50 ms latency, how many requests are in flight?
6. How does a read-heavy vs write-heavy ratio change your architecture?
7. Order these by speed and give magnitudes: RAM read, SSD read, same-DC round trip, cross-world round trip.

Hands-on exercises

1. Reproduce the “numbers every engineer should know” table from memory, then check it.
2. Round seconds-per-day to a power of ten, then recompute three QPS estimates using the rounded value.
3. Pick an app you use. Guess its DAU and requests/user/day, and compute average and peak QPS.
4. Estimate storage for a chat app handling 1 billion messages/day at ~300 bytes each, kept 1 year, replicated 3x.

5. Given $p_{50} = 20$ ms and $p_{99} = 200$ ms, reason about what a page that fans out to 50 services will likely experience.

Mini project

SIZE A SYSTEM ON ONE PAGE

Choose a well-known app (Twitter/X, WhatsApp, YouTube, an e-commerce site). On a single sheet, produce a full estimate using the six-step method: DAU, average and peak QPS, read:write ratio, storage per year (with replication), and bandwidth. Finish with a one-line conclusion naming the infrastructure the numbers demand (cache? CDN? sharding? queues?). This is exactly the opening move of a real design interview — practice it until it's fast.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE ESTIMATION

Open your design notebook. Pick a target scale (say 20M DAU). Using this chapter's method, compute: read and write QPS (average and peak), read:write ratio, storage per year for posts and media (with replication), cache working-set size, and egress bandwidth. Then write the “**numbers → architecture**” conclusions: what they imply for caching, database choice, sharding, and CDN. This estimate is the compass for every design decision you'll make in Parts II and III — keep it at the front of your notebook.

Chapter summary

- **Latency** is per-request time and is a **distribution** — describe it with percentiles ($p_{50}/p_{99}/p_{99.9}$). The **tail** dominates user experience, especially when requests fan out (“tail at scale”).
- **Throughput** (QPS) is requests per second; **Little's Law** (concurrency = throughput × latency) tells you how much is in flight. Throughput is capped by the **bottleneck** stage.
- As utilization nears 100%, latency **explodes** — so run with **headroom** (~50–70%), never at the redline.
- Know the **numbers** (RAM ~100 ns, SSD ~100 μ s, same-DC RTT ~0.5 ms, cross-world RTT ~150 ms) and the tricks (~100,000 seconds/day; powers of ten).
- **Back-of-the-envelope estimation** turns a user count into QPS, storage, and bandwidth — and those numbers **choose the architecture** (cache, CDN, sharding, queues) before you draw anything.

COMING NEXT — PART II BEGINS

Chapter 6 — The Client-Server Model & the Request Lifecycle. Part I gave you the foundations — hardware, networking, the web, and the numbers. Part II starts assembling real scalable systems. We begin with the fundamental shape of almost every system — clients and servers — and trace a request all the way through a modern multi-tier architecture, setting up load balancing and caching in the chapters that follow.